

|                                        |                    |                                                                                                                                          |
|----------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IFGOIANO – CAMPUS URUTAI<br/>SI</b> |                    |  <b>INSTITUTO FEDERAL<br/>GOIANO</b><br>Campus Urutai |
| <b>ED I</b>                            | <b>3º Matutino</b> | <b>Júnio César de Lima</b>                                                                                                               |
| <b>Trabalho de implementação</b>       |                    |                                                                                                                                          |

### Implementação 1

Criar uma lista simplesmente encadeada para armazenar informações sobre as notas de alunos da turma de ED1. Cada nodo da lista guarda a matrícula e a nota de um aluno, além da referência. Os alunos devem ficar ordenados seguindo a nota, onde a maior nota deve ser a primeira. Em caso de empate, os alunos serão classificados na ordem de inserção. O programa deve conter os seguintes métodos:

- Construtor;
- Inserção em ordem;
- Remoção dado a matrícula do aluno;
- Busca dados de aluno por nota;
- Busca dados de aluno por matrícula;
- Busca dados de aluno por cidade;
- Tamanho da lista.

O projeto deverá conter no mínimo 3 classes:

- Classe ListaAluno: contém as implementações da lista;
- Classe Aluno: define os objetos alunos; contendo os seguintes campos: matrícula, nome, curso, endereço e telefone;
- Classe TesteListaAluno: realiza os testes dos métodos.

### Implementação 2

Criar uma fila estática para armazenar informações sobre os clientes do Banco TeuDinheiroÉMeu. Cada nodo da fila guarda o código do cliente, além da referência. Os clientes devem ficar ordenados seguindo a ordem de inserção. O programa deve conter os seguintes métodos:

- Construtor;
- Inserção;
- Remoção;
- Busca dados de cliente por nome;
- Busca dados de cliente por código;
- Busca dados de cliente por cidade;
- Tamanho da fila.

O projeto deverá conter no mínimo 3 classes:

- Classe FilaCliente: contém as implementações da fila;
- Classe Cliente: define os objetos clientes contendo os seguintes campos: código, nome, data

- de nascimento, endereço e telefone;
- Classe **TesteFilaCliente**: realiza os testes dos métodos.

### Implementação 3

Implementar um programa em Java que seja capaz de receber uma expressão aritmética na forma infixa (notação tradicional) e realizar as seguintes etapas:

1. **Conversão para Pós-fixa (Notação Polonesa Reversa)**: Transformar a expressão infixa em uma expressão pós-fixa. A expressão pós-fixa é mais adequada para avaliação computacional.
2. **Avaliação da Expressão Pós-fixa**: Calcular o resultado da expressão pós-fixa utilizando uma pilha (stack) para armazenar os operandos e operadores.

Considerações importantes:

- O programa deve ser capaz de lidar com as quatro operações básicas (+, -, \*, /) e parênteses.
- A entrada da expressão infixa pode conter espaços em branco, mas o programa deve removê-los antes da conversão.
- O programa deve exibir o resultado da avaliação da expressão pós-fixa.

### Implementação 4

Implementar uma **Lista Duplamente Encadeada** para gerenciar o histórico de um "Player de Músicas". Diferente da lista simples, cada nodo deve possuir uma referência para o próximo elemento e outra para o elemento anterior, permitindo a navegação bidirecional.

O programa deve conter os seguintes métodos:

- **Construtor**;
- **Inserir Música**: Adicionar uma nova música ao final da lista;
- **Remover Música**: Remover uma música da lista dado o seu título;
- **Exibir Playlist**: Percorrer e imprimir a lista do início ao fim;
- **Exibir Playlist Reversa**: Percorrer e imprimir a lista do fim para o início (usando o ponteiro anterior);
- **Busca por Título**: Localizar os dados de uma música pelo nome;
- **Tamanho da Lista**: Retornar a quantidade de músicas na playlist.

O projeto deverá conter no mínimo 3 classes:

- **Classe ListaMusica**: contém as implementações da lista duplamente encadeada (referências primeiro e ultimo);
- **Classe Musica**: define os objetos músicas contendo os campos: título, artista, álbum e duração (em segundos);
- **Classe TestePlaylist**: realiza os testes dos métodos.

Considerações importantes:

- Cada nodo da lista (ou a classe Musica, se preferir usar o próprio objeto como nodo) deve obrigatoriamente conter dois ponteiros: proximo e anterior.
- A implementação deve garantir que, ao remover ou inserir elementos, ambos os ponteiros dos vizinhos sejam atualizados corretamente para evitar perda de integridade da lista.

### Implementação 5

Implementar um programa em Java que seja capaz de percorrer todas as cidades representadas no grafo a seguir usando uma busca em profundidade através do emprego de uma pilha:



## Implementação 7

Implementar os seguintes métodos de ordenação:

BubbleSort;  
SelectionSort;  
InsertionSort;

Criar dois conjuntos de vetores de dados (dataset) com valores aleatórios, sendo que o tamanho de cada vetor é dado a seguir de acordo com os dois conjuntos:

- a) Dataset1: 10 – 20 – 30 – 40 – 50 – 60 – 70 – 80 – 90 - 100
- b) Dataset2: 1000 - 2000 – 3000 – 4000 – 5000 – 6000 – 7000 – 8000 – 9000 - 10000

Cada método de ordenação deve receber os 20 vetores como entrada (receber sempre os mesmos vetores) e devolver o vetor ordenado. Para cada execução tomar o tempo de execução (calcular a média de 100 execuções).

O projeto deverá conter no mínimo 4 classes:

- Classe Ordena: contém as implementações dos algoritmos de ordenação;
- Classe Item: contém o objeto a ser ordenado;
- Classe Utilidades: contém a geração dos vetores, leitura e escrita de arquivos e demais utilidades;
- Classe TesteOrdena: realiza a tomada de tempo.

## Implementação 8

Implementar os seguintes métodos de ordenação:

BubleSort;  
InsertionSort;  
ShellSort;

Criar dois conjuntos de vetores de dados (dataset) com valores aleatórios, sendo que o tamanho de cada vetor é dado a seguir de acordo com os dois conjuntos:

- c) Dataset1: 10 – 20 – 30 – 40 – 50 – 60 – 70 – 80 – 90 - 100
- d) Dataset2: 1000 - 2000 – 3000 – 4000 – 5000 – 6000 – 7000 – 8000 – 9000 - 10000

Cada método de ordenação deve receber os 20 vetores como entrada (receber sempre os mesmos vetores) e devolver o vetor ordenado. Para cada execução tomar o tempo de execução (calcular a média de 100 execuções).

O projeto deverá conter no mínimo 4 classes:

- Classe Ordena: contém as implementações dos algoritmos de ordenação;
- Classe Item: contém o objeto a ser ordenado;
- Classe Utilidades: contém a geração dos vetores, leitura e escrita de arquivos e demais utilidades;
- Classe TesteOrdena: realiza a tomada de tempo.

## Implementação 9

Implementar os seguintes métodos de ordenação:

ShellSort;  
QuickSort;  
MergeSort;

Criar dois conjuntos de vetores de dados (dataset) com valores aleatórios, sendo que o tamanho de cada vetor é dado a seguir de acordo com os dois conjuntos:

- e) Dataset1: 1000 - 2000 – 3000 – 4000 – 5000 – 6000 – 7000 – 8000 – 9000 - 10000
- f) Dataset2: 10000 - 20000 – 30000 – 40000 – 50000 – 60000 – 70000 – 80000 – 90000 - 100000

Cada método de ordenação deve receber os 20 vetores como entrada (receber sempre os mesmos vetores) e devolver o vetor ordenado. Para cada execução tomar o tempo de execução (calcular a média de 100 execuções).

O projeto deverá conter no mínimo 4 classes:

- Classe Ordena: contém as implementações dos algoritmos de ordenação;
- Classe Item: contém o objeto a ser ordenado;
- Classe Utilidades: contém a geração dos vetores, leitura e escrita de arquivos e demais utilidades;
- Classe TesteOrdena: realiza a tomada de tempo.

## Implementação 10

Implementar os seguintes métodos de ordenação:

SelectionSort;  
InsertionSort;  
QuickSort;

Criar dois conjuntos de vetores de dados (dataset) com valores aleatórios, sendo que o tamanho de cada vetor é dado a seguir de acordo com os dois conjuntos:

- g) Dataset1: 1000 - 2000 – 3000 – 4000 – 5000 – 6000 – 7000 – 8000 – 9000 - 10000
- h) Dataset2: 10000 - 20000 – 30000 – 40000 – 50000 – 60000 – 70000 – 80000 – 90000 - 100000

Cada método de ordenação deve receber os 20 vetores como entrada (receber sempre os mesmos vetores) e devolver o vetor ordenado. Para cada execução tomar o tempo de execução (calcular a média de 100 execuções).

O projeto deverá conter no mínimo 4 classes:

- Classe Ordena: contém as implementações dos algoritmos de ordenação;
- Classe Item: contém o objeto a ser ordenado;
- Classe Utilidades: contém a geração dos vetores, leitura e escrita de arquivos e demais utilidades;
- Classe TesteOrdena: realiza a tomada de tempo.

## Orientações de Entrega e Apresentação

- **GitHub:** Postar o projeto com documentação completa (explicação do problema, da solução e das saídas). Caso haja comparações, incluir e explicar os gráficos gerados.
- **Moodle:** Cada membro do grupo deve realizar o envio individual até o dia **08/06/2026**.
- **Apresentação:** Cada grupo terá o tempo máximo de **15 minutos**.
- **Padronização:** Todas as classes Java devem conter um cabeçalho com:
  - Nome completo dos alunos;

- Nome da disciplina e data de codificação;
- Explicação do objetivo da classe.